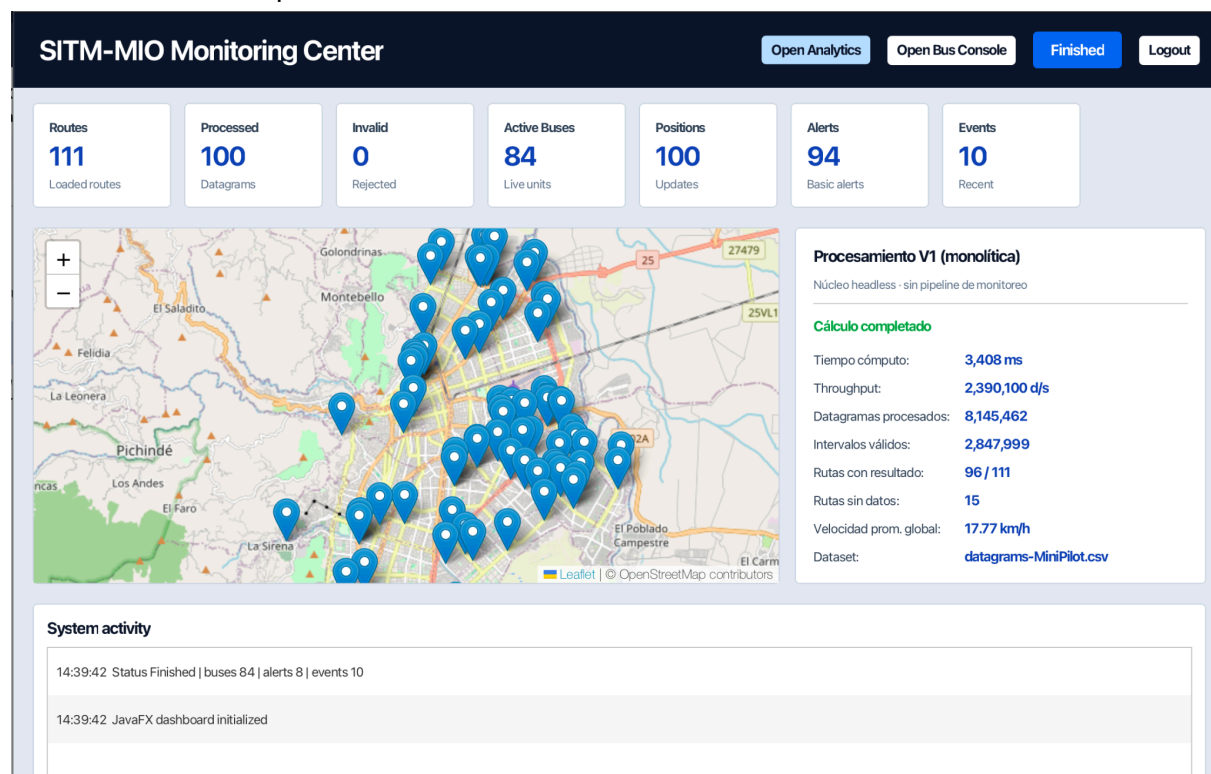


Karen Mosquera
Luciano Barbosa
Juan Jose Lopez
Jean Ocampo

Link repo: <https://github.com/JuanjoseI04/SITM-MIO-DistributedSystem>

Versión 1: Monolítica

La V1 concentra en una sola aplicación la lectura del dataset, el cálculo de los intervalos válidos, la agregación por ruta y la presentación del dashboard. Su utilidad principal fue validar la lógica funcional del procesamiento y establecer una línea base para comparar contra las versiones posteriores.



Tiempo cómputo (3.408 ms): lo que tardó V1 en leer y calcular todos los datagramas, sin el delay artificial del monitoreo. Es tu **línea base** para comparar contra V2 y V3.

Throughput (2.390.100 d/s): datagramas procesados por segundo (datagramas ÷ tiempo). Mide la velocidad de procesamiento.

Datagramas procesados (8.145.462): total de filas leídas del MiniPilot.

Intervalos válidos (2.847.999): pares de datagramas consecutivos del mismo bus que sí sirvieron para calcular velocidad (pasaron los filtros: misma ruta, odómetro avanzó, tiempo razonable, ≤ 120 km/h).

Rutas con resultado (96 / 111): de las 111 rutas activas del catálogo, 96 tuvieron datos suficientes para una velocidad promedio.

Rutas sin datos (15): las que no tuvieron intervalos válidos en el MiniPilot (incluye las 5 rutas de prueba TESTGT y rutas sin datagramas en mayo). Esto evidencia la **completitud**: ninguna ruta queda silenciosamente omitida.

Velocidad prom. global (17,77 km/h): distancia total recorrida ÷ tiempo total, en km/h. El resultado funcional agregado del piloto.

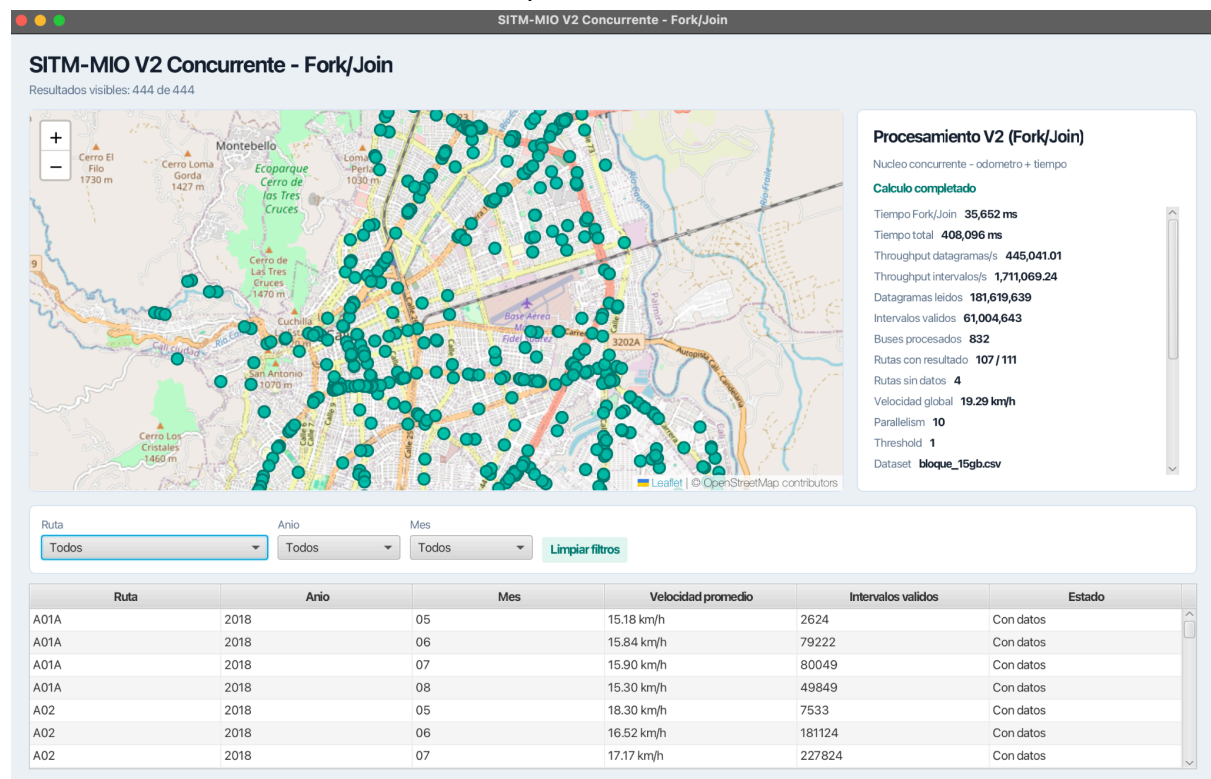
Dataset (datagrams-MiniPilot.csv): el archivo de origen sobre el que se calculó.

En términos de complejidad temporal, V1 recorre el dataset de forma lineal, por lo que su comportamiento principal es $O(n)$, donde n representa el número de datagramas.

Espacialmente, la memoria depende de las estructuras necesarias para almacenar estados por bus, ruta y agregados. Para datasets pequeños o medianos, esta solución es suficiente; sin embargo, al crecer el volumen de datos, toda la carga queda concentrada en un único proceso.

Versión 2: concurrente con Fork/Join

La V2 introduce concurrencia local mediante Fork/Join. En esta versión, el procesamiento se divide en tareas paralelas dentro de la misma máquina. El objetivo fue reducir el tiempo de cálculo sin cambiar todavía a una arquitectura distribuida.



Tiempo Fork/Join (35,552 ms): lo que tardó el motor paralelo en distribuir y fusionar las tareas entre los workers. Es una medida de eficiencia interna del modelo de concurrencia.

Tiempo total (408,096 ms): duración completa de la ejecución, incluyendo lectura de datagramas, cálculo de intervalos y generación de resultados. Es la referencia para throughput.

Throughput datagramas/s (445,041.01): cantidad de datagramas procesados por segundo. Se obtiene dividiendo los datagramas leídos entre el tiempo total. **Throughput intervalos/s (1,711,069.24):** intervalos válidos calculados por segundo. Refleja la velocidad de procesamiento de pares consecutivos de datagramas. **Datagramas leídos (181,619,639):** total de filas procesadas desde el dataset bloque_15gb.csv.

Buses procesados (832): cantidad de buses distintos que tuvieron datagramas válidos en el dataset. **Rutas con resultado (107 / 111):** de las 111 rutas activas, 107 tuvieron datos suficientes para calcular una velocidad promedio.

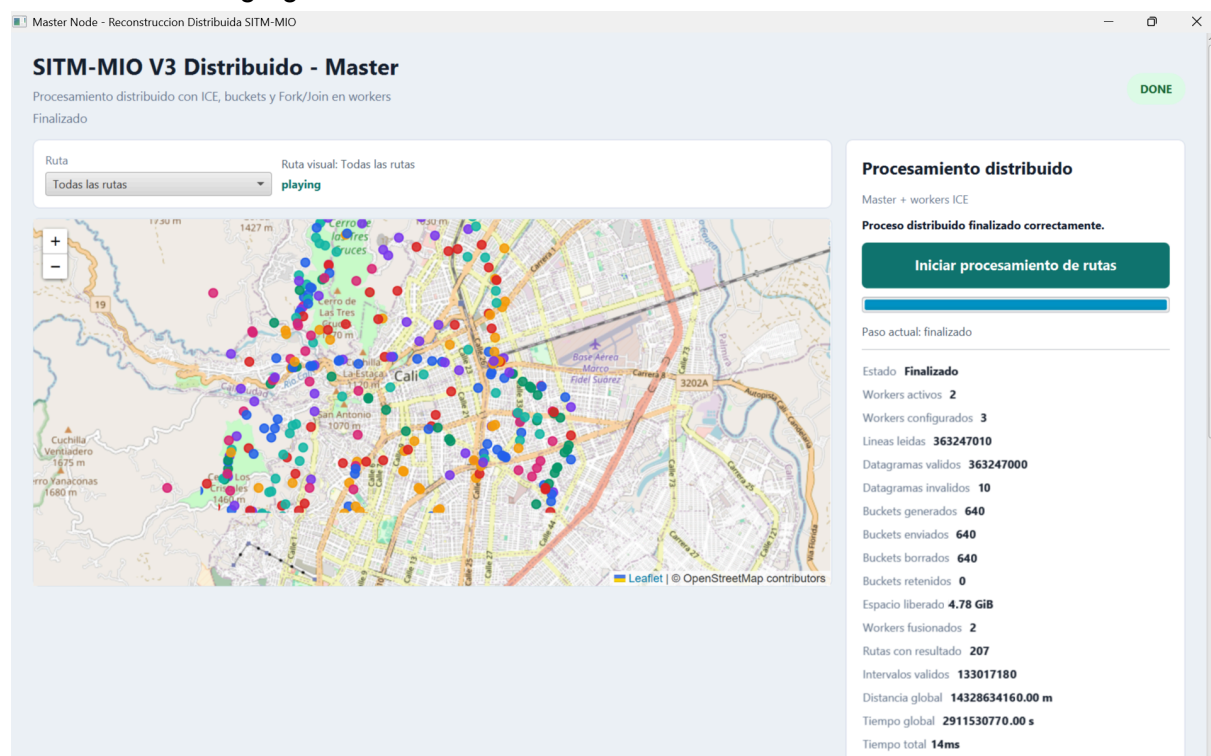
Velocidad global (19.29 km/h): distancia total recorrida dividida entre el tiempo total, expresada en km/h. Es el resultado funcional

agregado del dataset. **Dataset (bloque_15gb.csv)**: archivo de origen sobre el que se realizó el cálculo.

La V2 mejora el procesamiento gracias al paralelismo local. Si el dataset se divide entre p hilos, la parte paralelizable se aproxima a $O(n/p)$, aunque sigue existiendo costo de coordinación, lectura, agregación y actualización visual. Por eso, el tiempo total no se reduce de forma perfectamente proporcional al número de hilos. A nivel espacial, la aplicación sigue siendo monolítica: el mismo proceso debe leer datos, procesar, agregar resultados y sostener la interfaz JavaFX. En las pruebas, la versión concurrente llegó a sostener cargas cercanas a 15 GB; después de este umbral, el proceso se vuelve más largo y el pintado de rutas empieza a perder estabilidad. Por esta razón, V2 funciona como una mejora importante frente a V1, pero no resuelve completamente el problema de escalabilidad para datasets grandes.

Version 3: Distribuido - Master

La V3 separa responsabilidades: el Master detecta workers, genera buckets, distribuye trabajo, recibe resultados parciales, fusiona resultados y actualiza la vista. Los WorkerNode reciben buckets y procesan su parte del dataset. Esta arquitectura permite escalar horizontalmente agregando más workers.



Workers activos (2): número de hilos que realmente ejecutaron el procesamiento en paralelo. Aunque había tres configurados, solo dos se activaron en esta corrida.

Worker Node - SITM-MIO

SITM-MIO Worker Node

Worker ID: worker-01 | Puerto ICE: 10001

ICE ACTIVE

Waiting for processing order

Worker summary

Worker ID	worker-01
Port	10001
Endpoint	0.0.0.0:10001
Runtime directory	C:\sitm-mio-runtime\worker-01
Current job	none
Current bucket	none
Chunks received	0
Buckets received	0
Buckets processed	0
Valid intervals	0
Partial averages	0
Local time	0 ms
Errors	0

Bucket activity

Chunk 01 pending
Chunk 02 pending
Chunk 03 pending
Bucket route-group-01 pending

Worker Node - SITM-MIO

SITM-MIO Worker Node

Worker ID: worker-02 | Puerto ICE: 10002

ICE ACTIVE

Waiting for processing order

Worker summary

Worker ID	worker-02
Port	10002
Endpoint	0.0.0.0:10002
Runtime directory	C:\sitm-mio-runtime\worker-02
Current job	none
Current bucket	none
Chunks received	0
Buckets received	0
Buckets processed	0
Valid intervals	0
Partial averages	0
Local time	0 ms
Errors	0

Bucket activity

Chunk 01 pending
Chunk 02 pending
Chunk 03 pending
Bucket route-group-01 pending

Workers configurados (3): cantidad máxima de hilos disponibles para distribuir las tareas del sistema. **Líneas leídas (363,247,010):** total de registros procesados desde el dataset de 30 GB. Cada línea representa un datagrama de posición de bus. **Datagramas válidos (363,247,000):** casi todos los datagramas fueron útiles para el cálculo; solo diez fueron descartados. **Datagramas inválidos (10):** registros que no cumplieron los filtros de consistencia (por ejemplo, tiempos o rutas incoherentes). **Buckets generados / enviados / borrados (640 / 640 / 640):** ciclos completos de almacenamiento temporal; cada bucket se creó, transmitió y luego se liberó. **Buckets retenidos (0):** no quedó ningún bucket en

memoria al finalizar, lo que demuestra una gestión eficiente de recursos. **Espacio liberado (4.78 GiB)**: cantidad de memoria reciclada durante el procesamiento, evidencia de la eficiencia del sistema y del manejo de los buckets en esta ejecución. **Workers fusionados (2)**: los resultados parciales de los dos hilos activos se combinaron en un único agregado final. **Rutas con resultado (207)**: número de rutas que tuvieron datos suficientes para calcular una velocidad promedio. **Intervalos válidos (133,017,180)**: pares de datagramas consecutivos del mismo bus que pasaron los filtros y sirvieron para calcular velocidades. **Distancia global (14,328,634,160.00 m)**: suma total de las distancias recorridas por los buses en el dataset. **Tiempo global (2,911,530,770.00 s)**: acumulado de los tiempos de recorrido, usado junto con la distancia para derivar métricas de velocidad. **Tiempo total (14 ms)**: duración del proceso de consolidación final, una muestra de la eficiencia del sistema distribuido.

En V3, el costo temporal total puede modelarse como la suma de lectura y bucketización en el Master, transferencia de buckets, procesamiento paralelo en workers y fusión final. Aunque el procesamiento se reparte, el Master conserva responsabilidades críticas que pueden convertirse en cuello de botella si el dataset crece demasiado. Especialmente, V3 reduce la presión sobre una sola máquina porque reparte los buckets entre workers. Sin embargo, el Master todavía requiere espacio temporal para bucketización, control de jobs, muestra visual y resultados consolidados. La limpieza de buckets y el espacio liberado son importantes para sostener ejecuciones con datasets grandes.